4. 📘 OPTIMIZACIÓN DE CONSULTAS DISTRIBUIDAS – PARTE 1. EL OPTIMIZADOR CENTRALIZADO

4. 📘 OPTIMIZACIÓN DE CONSULTAS DISTRIBUIDAS – PARTE 1. EL OPTIMIZADOR CENTRALIZADO 1
4.1. 🧭 Introducción 3
4.2. ⚙️ Arquitectura del optimizador basado en costos 5
4.2.1. 🔄 Transformador de consultas 6
4.2.2. 🧱 Estimador 7
4.2.3. 🌳 Generador de planes 9
4.3. 🗺️ Planes de ejecución 10
4.3.1. 🛠️ Obtención de un plan con explain plan 11
4.3.1.1. Caso de estudio: control médico 12
4.3.2. 📖 Lectura de un plan 14
4.3.2.1. Predicados: access y filter 14
4.3.2.2. Orden de ejecución: primero la primera hija 15
Ejercicio en clase 1 - Índice sobre consultorio 15
4.3.2.3. La sección Note 15
4.3.3. 🔁 Plan real con display_cursor 16
4.4. 🧭 Métodos de acceso a datos (access paths) 18
4.4.1. 📑 El índice como fuente de rowids 19
4.4.2. 🗂️ Acceso a tablas 20
4.4.2.1. Table access full 20
4.4.2.2. Table access by rowid 24
4.4.2.3. Sample table scan 25
4.4.2.4. In-memory table scan 26
4.4.3. 🌲 Acceso a índices B-tree 27
4.4.3.1. Index unique scan 27
Ejercicio en clase 2 - Índice sobre una columna con nulos 28
4.4.3.2. Index range scan 29
Ejercicio en clase 3 - Ordenamiento por la llave primaria 31
4.4.3.3. Index full scan 32
4.4.3.4. Index fast full scan 33
4.4.3.5. Index skip scan 34
4.4.3.6. Index join scan 36
Ejercicio en clase 4 - Selección del método de acceso 37

4.4.4. 💰 Desglose del costo: I/O y CPU 37
4.5. 📚 Referencias 42

El tema se divide en cuatro partes. Las tres primeras estudian el optimizador de una base de datos centralizada, y la cuarta reutiliza esos conceptos para entender los algoritmos de un optimizador global en una base de datos distribuida, donde aparece un componente que el optimizador centralizado no tiene: el costo de comunicación entre sitios.

Parte	Contenido
Parte 1 (este documento)	Arquitectura del optimizador, planes de ejecución, métodos de acceso y desglose del costo.
Parte 2	Joins: orden, métodos y tipos.
Parte 3	Estadísticas, histogramas y factor de selectividad.
Parte 4	Optimización de consultas distribuidas.

4.1. 🧭 Introducción

¿Por qué no basta con la consulta reducida que se obtuvo en el Tema 3?
En el Tema 3 se trabajó con el esquema global y los esquemas de fragmentación. Mediante propiedades, reglas de transformación y equivalencias del álgebra relacional se simplificó una consulta distribuida, eliminando las expresiones que producen conjuntos vacíos. El resultado fue la **consulta reducida** (o consulta fragmentada).
Sin embargo, ese proceso deja fuera dos aspectos determinantes para el desempeño:

- El orden de ejecución de las operaciones. Una misma consulta admite muchos árboles de operadores equivalentes, y su costo puede diferir en órdenes de magnitud. El Tema 3 dejó mencionada la explosión combinatoria de estos árboles; en este tema se estudia cómo la enfrenta el optimizador.
- Las características de los datos. Cardinalidad de cada fragmento, selectividad de los predicados, existencia de índices, distribución de los valores, etc. El álgebra relacional es indiferente a ellas, pero el costo real no.

Por lo anterior, se requiere una etapa adicional encargada de generar los llamados **planes de ejecución** y de seleccionar aquel que represente la opción óptima o, por lo menos, la más cercana a lo óptimo. En una base de datos distribuida esta etapa corresponde al **optimizador global**, el cual debe realizar tareas adicionales a las de un optimizador centralizado.

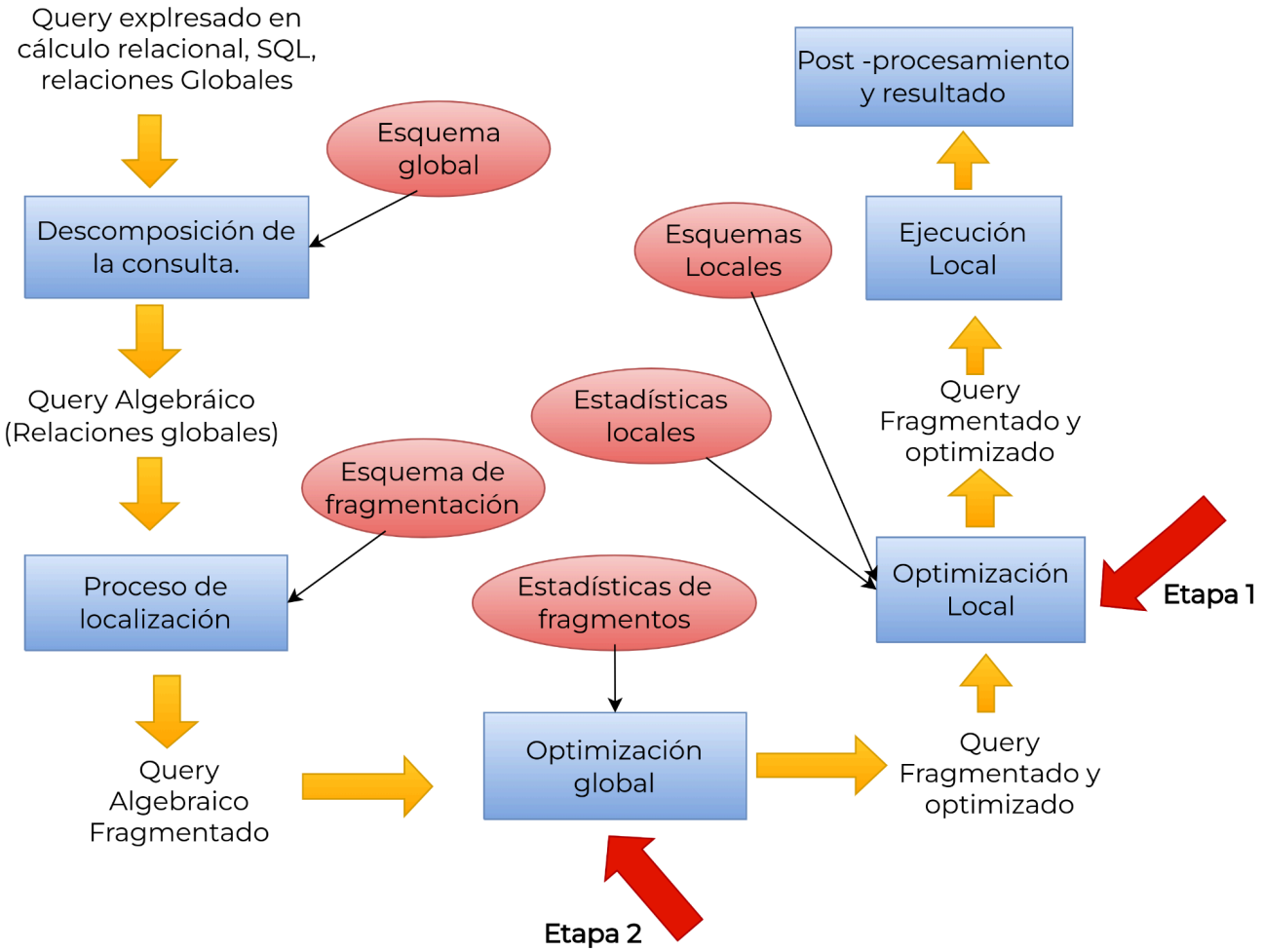


Figura 01. *Etapas del procesamiento de consultas distribuidas: la consulta global se descompone (esquema global), se localiza (esquema de fragmentación) y se optimiza globalmente (estadísticas de fragmentos); cada sitio la optimiza localmente (esquemas y estadísticas locales), la ejecuta y entrega el resultado al post-procesamiento. Se señalan la etapa 1 (optimización local) y la 2 (global).*

Para comprender esos algoritmos, el tema se aborda en dos etapas, señaladas en la figura anterior:

A. Etapa 1 (partes 1 a 3). Características de un optimizador centralizado. Corresponde a la caja de *optimización local* que ejecuta cada sitio.
B. Etapa 2 (parte 4). Con los conceptos de la etapa 1 se estudian los algoritmos del *optimizador global*.

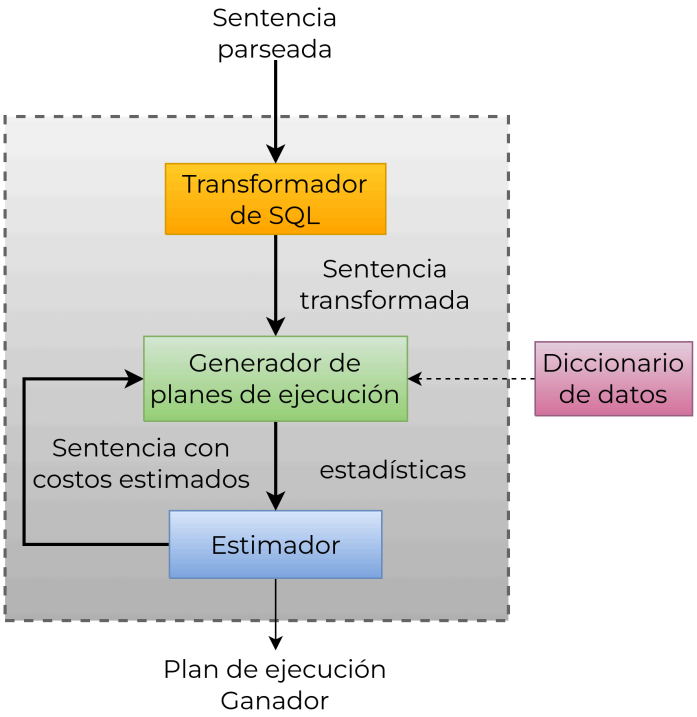
📝 **Nota**
🌐 **Modernización: el núcleo centralizado sigue vigente en los sistemas distribuidos.** Los motores distribuidos actuales no reemplazan al optimizador centralizado: lo extienden. En Oracle, una consulta que referencia tablas remotas a través de *database links* la procesa el mismo optimizador basado en costos del sitio local, que agrega al plan operaciones `REMOTE` cuyo SQL se ejecuta en otro sitio (se estudia en la parte 4). En PostgreSQL, el *foreign data wrapper* `postgres_fdw` permite que el planificador local, con la opción `use_remote_estimate`, solicite al servidor remoto un `explain` para estimar el costo de la parte remota antes de elegir el plan. En ambos casos, entender el optimizador local es requisito para entender el distribuido.

4.2. ⚙️ Arquitectura del optimizador basado en costos

¿Qué ocurre con una sentencia `select` entre el momento en que se analiza su sintaxis y el momento en que comienza a leer datos?
La entrada del optimizador es una sentencia SQL ya analizada (*parsed*), formada por un conjunto de **bloques de consulta** (*query blocks*): cada instrucción `select` representa un bloque. Por ejemplo, la siguiente consulta contiene dos bloques:

```
select first_name, last_name
from hr.employees
where department_id in (
  select department_id
  from hr.departments
  where location_id = 1800
);
```

Por cada bloque, el optimizador genera un **subplan** de ejecución. El número de subplanes posibles crece con el número de tablas listadas en la cláusula `from` de cada bloque.
El optimizador está formado por tres componentes:
[[keep: optimizador]]



[[/keep]]

Figura 02. *Componentes del optimizador basado en costos: la sentencia parseada pasa por el transformador, que entrega la sentencia transformada al generador de planes; este consulta las estadísticas del diccionario de datos y envía cada plan al estimador, que le regresa su costo, en un ciclo cuya salida es el plan de ejecución ganador.*

A. Transformador de consultas (*query transformer*): decide si conviene reescribir la sentencia para obtener mejores planes.
B. Generador de planes (*plan generator*): construye planes alternativos para cada bloque.
C. Estimador (*estimator*): asigna un costo numérico a cada plan.

El generador y el estimador trabajan en ciclo: cada plan candidato se envía al estimador y su costo regresa al generador, que conserva el de menor costo. Al terminar, el resultado es el **plan de ejecución ganador**.

4.2.1. 🔄 Transformador de consultas

Debido a que SQL es un lenguaje declarativo y no procedimental, el optimizador es libre de dividir, reorganizar o reescribir una sentencia y procesarla en cualquier orden, siempre que el resultado sea equivalente. Algunas transformaciones que aplica la base de datos:

Transformación	Idea principal
OR expansion	Convierte un predicado con `or` en varias consultas combinadas con `union all`, cada una

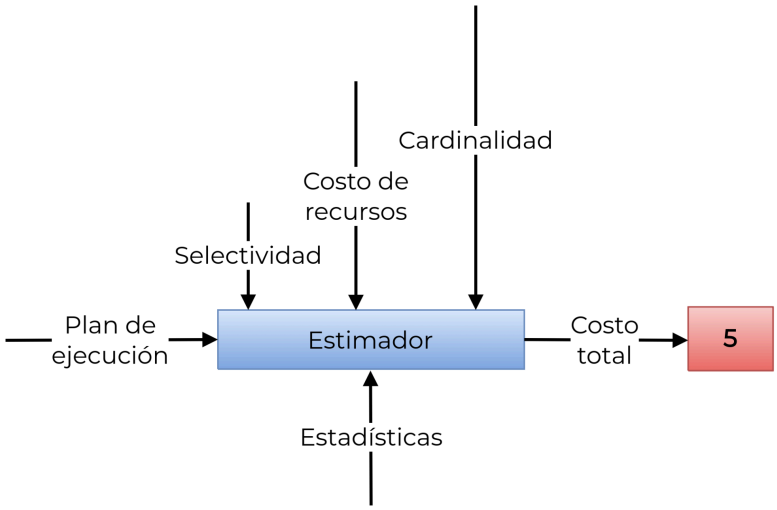
Transformación	Idea principal
	con la posibilidad de usar su propio índice.
View merging	Fusiona una vista o subconsulta en el `from` con la consulta externa para optimizarlas como un solo bloque.
Predicate pushing	Empuja un predicado de la consulta externa hacia el interior de una vista que no pudo fusionarse.
Subquery unnesting	Convierte una subconsulta (por ejemplo, con `in` o `exists`) en un join o semijoin.
Query rewrite with materialized views	Reescribe la consulta para leer una vista materializada que ya contiene el resultado o parte de él.
Star transformation	Optimiza consultas sobre esquemas en estrella (tabla de hechos y dimensiones).
In-memory aggregation	Optimiza agregaciones sobre tablas en el *IM column store* (solo mención; su práctica pertenece a BDA).
Table expansion	Permite usar planes distintos por partición cuando un índice existe solo en algunas particiones.
Join factorization	Factoriza las tablas comunes de las ramas de un `union all` para leerlas una sola vez.

En el ejemplo anterior, *subquery unnesting* puede transformar la subconsulta con `in` en un semijoin entre `employees` y `departments`, con lo cual ambos bloques se optimizan juntos y el generador puede considerar más órdenes y métodos de join.
Algunas transformaciones se aplican siempre que son válidas, porque prácticamente nunca empeoran el plan (**transformaciones heurísticas**); otras se aplican solo si el estimador confirma que el plan transformado es más barato (**transformaciones basadas en costos**). Esta distinción retoma la que el Tema 3 presentó con el *predicate pushdown* de CockroachDB, considerado una transformación independiente del costo.

📝 **Nota**
🔗 **Gancho hacia la parte 4.** La transformación *view merging* es conveniente en una base centralizada, pero en una consulta distribuida puede deshacer una subconsulta que agrupaba deliberadamente tablas del mismo sitio remoto (*collocated inline view*). En la parte 4 se verá cómo evitarlo con el hint `no_merge`.

4.2.2. 🧱 Estimador

[[keep: estimador]]



[[/keep]]

Figura 03. *Entradas y salida del estimador: recibe un plan de ejecución y, con la selectividad, la cardinalidad, el costo de recursos y las estadísticas, calcula su costo total, un valor numérico (5 en el ejemplo).*

La función principal del estimador es determinar el **costo total**, un valor numérico, de un plan de ejecución determinado. En un optimizador centralizado, el costo se compone principalmente de **operaciones de I/O** y **uso de procesador**; en una base de datos distribuida se agrega el **costo de comunicación**, que se formaliza en la parte 4.
El estimador emplea tres medidas:

- Selectividad. Fracción estimada de registros que se obtendrían de una fuente de datos (una tabla, una vista o el resultado de un join) al aplicarle un predicado. Se expresa como un número en el rango [0, 1].
- Cardinalidad. Número estimado de registros que produce cada operación del plan. Se obtiene, en general, como la cardinalidad de la entrada multiplicada por la selectividad del predicado.
- Costo. Estimación de los recursos que consumirá cada operación: lecturas a disco, procesador y memoria.

Los valores de selectividad y cardinalidad se obtienen a partir de:

- Estadísticas de tablas, columnas e índices almacenadas en el diccionario de datos.
- Estadísticas dinámicas (*dynamic sampling*), obtenidas al momento de optimizar mediante un muestreo, cuando no existen estadísticas o son insuficientes.
- Histogramas, necesarios cuando la distribución de los valores de una columna no es uniforme.

Las estadísticas y los histogramas se estudian a detalle en la parte 3.

💻 Ejemplo

Suponer que se desea obtener a todos los empleados que son *managers*:

- Si las estadísticas indican que el 80 % de los empleados son *managers*, la selectividad es baja (0.8) y el optimizador seleccionaría un *full table scan*: leer la tabla completa es más barato que recorrer el índice y después visitar casi todos los bloques de la tabla, uno por uno.
- Si las estadísticas indican que solo el 5 % son *managers*, la selectividad es alta (0.05) y el optimizador seleccionaría un acceso por índice.

La consulta es la misma; lo que cambia la decisión son los datos.

4.2.3. 🌳 Generador de planes

El generador de planes construye y analiza los posibles planes de ejecución de cada bloque. Cuando una sentencia involucra varias tablas, el generador debe decidir la forma más eficiente de combinarlas, lo que implica seleccionar:

A. El método de acceso (*access path*) para cada tabla: leerla completa, usar un índice, etc. Se estudia en la sección de métodos de acceso.
B. El orden de los joins: qué pareja de tablas se combina primero y qué forma tiene el árbol de joins.
C. El método de join: *nested loops*, *hash join*, *sort merge join* o producto cartesiano.
D. El tipo de join indicado en la sentencia: *inner*, *outer*, semijoin o antijoin, que restringe los órdenes válidos.

Los puntos 2 a 4 se estudian en la parte 2.
La selección del mejor plan depende de varios factores:

- La forma en la que se escribe la consulta.
- El tamaño de los conjuntos de datos.
- Las estructuras de acceso existentes (índices, clústeres, particiones).
- Los recursos del sistema disponibles: memoria y procesador.
- Las estadísticas disponibles y qué tan actualizadas están.

Un principio general guía la construcción de planes con joins: **reducir el número de registros desde las primeras operaciones**, para que las siguientes sean menos costosas. Por ejemplo, conviene ejecutar primero los joins que producen a lo más un registro, como los que involucran un predicado de igualdad sobre una columna con restricción `primary key` o `unique` (`where empleado_id = 10`).
Para una sentencia con *n* tablas, el número de órdenes de join posibles crece de forma factorial, por lo que el generador no puede evaluar todos los planes en consultas grandes: emplea reglas de poda y cortes de búsqueda. Este problema es justamente el que la parte 4 clasifica en tipos de optimizadores (búsqueda exhaustiva, heurísticos, estáticos, dinámicos e híbridos).

📝 **Nota**

🌐 **Modernización: arquitecturas de optimizadores en sistemas actuales.** La separación entre transformación, generación y estimación es común a casi todos los optimizadores basados en costos, aunque la organización interna varía. Varios motores modernos se basan en dos marcos propuestos por Goetz Graefe, en los que el optimizador se define como un conjunto de reglas que transforman expresiones del álgebra y una estructura llamada *memo* agrupa las alternativas equivalentes para no recalcularlas:

- Volcano (1993) busca en dos fases: primero aplica todas las reglas de transformación hasta generar todas las expresiones lógicas equivalentes y, después, recorre ese espacio de arriba hacia abajo eligiendo los algoritmos físicos y podando por costo. Su debilidad es la primera fase: genera muchas alternativas que nunca llegan a costearse.
- Cascades (1995) intercala ambas fases: explora las alternativas lógicas de un grupo solo cuando la optimización las necesita, poda por costo desde el inicio, organiza la búsqueda como una pila de tareas pequeñas y asigna prioridades a las reglas para aplicar primero las más prometedoras.

Síntesis: Volcano explora primero todo el espacio lógico y después lo costea; Cascades lo explora bajo demanda, guiado por costos y con prioridades entre reglas, lo que lo hace más eficiente en consultas con muchas tablas.

Marco	Sistemas actuales que lo emplean
Volcano	Apache Calcite (`VolcanoPlanner`), usado como optimizador por motores como Apache Hive, Apache Flink y Apache Druid; versiones recientes de Calcite agregaron una búsqueda *top-down* inspirada en Cascades
Cascades	Microsoft SQL Server, Orca (Greenplum y VMware Tanzu), CockroachDB

El optimizador de Oracle no sigue ninguno de los dos marcos: separa la etapa de transformaciones (heurísticas y basadas en costos) de la de generación de planes por bloque. No confundir el marco de optimización Volcano con el *modelo de ejecución Volcano* (iteradores `open`/`next`/`close`), del mismo autor pero sin relación con el optimizador.

4.3. 🗺️ Planes de ejecución

¿Cómo se puede saber qué decidió el optimizador, y por qué una consulta tarda más de lo esperado?

Un **plan de ejecución** es la combinación de pasos que la base de datos realiza para ejecutar una sentencia SQL. Se representa como un árbol de operaciones, llamado árbol

de *row sources*: cada operación recibe registros de sus hijas, los procesa y los entrega a su padre. Un plan incluye:

- El método de acceso (*access path*) de cada tabla.
- El orden en el que se ejecutan los joins.
- El método seleccionado para cada join.
- Operaciones sobre los datos: filtrado, ordenamiento, agregación.
- La cardinalidad y el costo estimados de cada operación.
- Información de particionamiento, en tablas particionadas (columnas `Pstart` y `Pstop`).
- Información de procesamiento en paralelo.

Es importante distinguir dos tipos de plan:

- Plan estimado (prospectivo). Es el que el optimizador *elegiría* para la sentencia, sin ejecutarla. Se obtiene con `explain plan`.
- Plan real (ejecutado). Es el que *efectivamente* se usó al ejecutar la sentencia y se encuentra en memoria (*cursor cache*). Se obtiene con `dbms_xplan.display_cursor`.

Ambos pueden diferir: entre otras razones, `explain plan` no considera los valores concretos de las variables *bind* ni los ajustes que ocurren durante la ejecución.

4.3.1. 🛠️ Obtención de un plan con explain plan

Al ejecutar `explain plan`, el optimizador elige un plan para la sentencia, sin ejecutarla, e inserta los datos que lo describen en la tabla `plan_table`.
En versiones actuales, `plan_table` ya existe para todos los usuarios como un sinónimo público hacia una tabla temporal global del diccionario, por lo que no es necesario crearla. Los scripts `utlxplan.sql` (crea una `plan_table` propia) y `catplan.sql` se ubican en `$ORACLE_HOME/rdbms/admin` y solo son necesarios en escenarios particulares.
Para mostrar el contenido de `plan_table` se emplea la función `dbms_xplan.display`, que recibe tres parámetros: el nombre de la tabla del plan, el identificador de la sentencia (`statement_id`) y el formato de salida. Los formatos más comunes son:

[[keep: formatos-plan]]

Formato	Contenido
`basic`	Solo identificador, operación y nombre del objeto.
`typical`	Formato por omisión: agrega filas, bytes, costo, tiempo y predicados.
`serial`	Como `typical`, sin la información de ejecución en paralelo.
`all`	Como `typical`, más proyección de columnas y alias de los bloques de consulta.

[[/keep]]

También existen los scripts `utlxpls.sql` (plan serial) y `utlxplp.sql` (plan con información de paralelismo) en `$ORACLE_HOME/rdbms/admin`, que muestran el último plan almacenado en `plan_table`.

4.3.1.1. Caso de estudio: control médico

Para los ejemplos de esta parte se emplea el modelo relacional de un sistema de control de citas y médicos de un hospital. Los scripts para crearlo se encuentran en la carpeta compartida del tema: descomprimir `ejemplo-control-medico.zip` y ejecutar `s-01-control-medico-main.sql`.

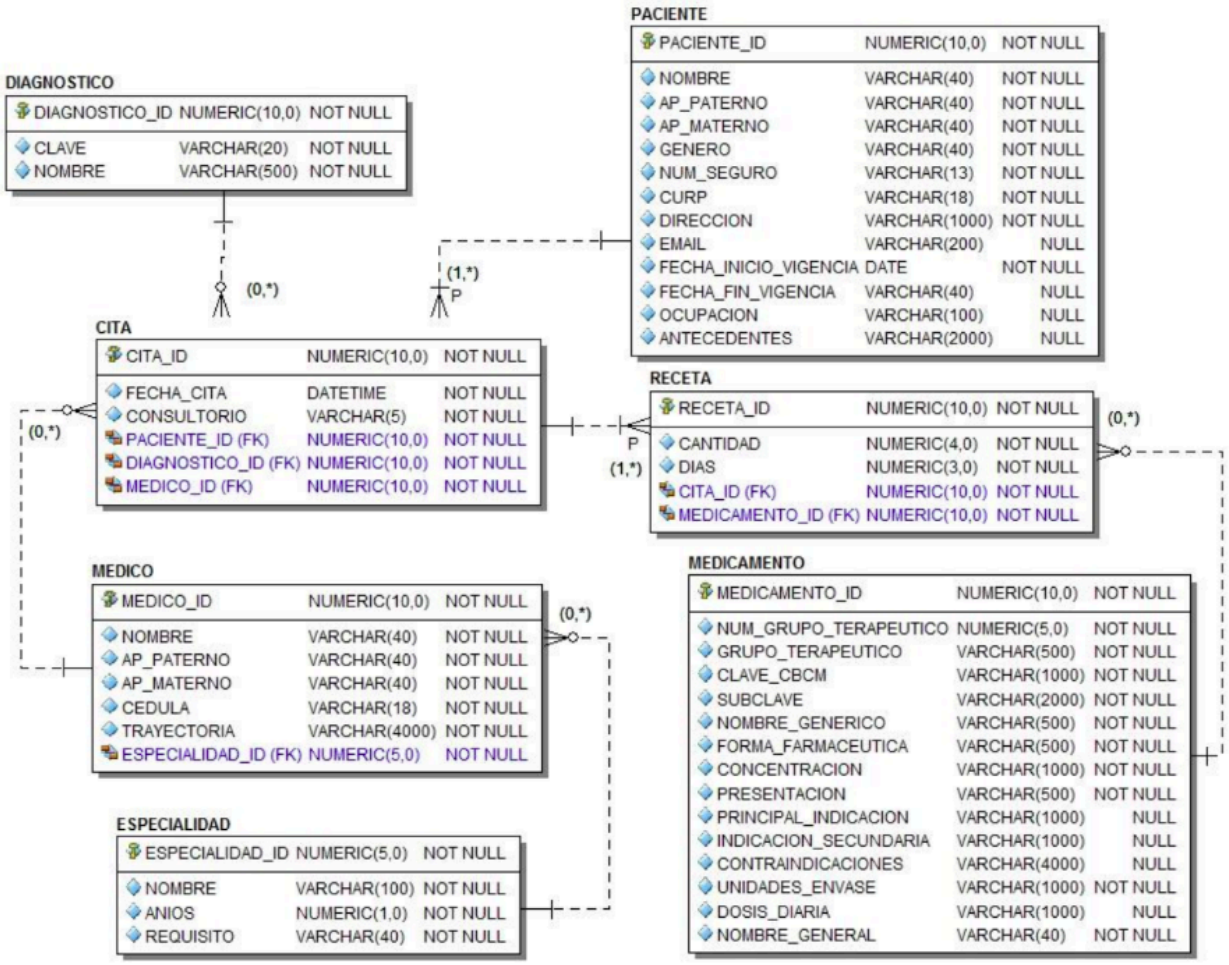


Figura 04. *Modelo relacional del caso de estudio control médico, con siete tablas: paciente, medico, especialidad, diagnostico, cita, receta y medicamento. Una cita referencia a un paciente, un médico y un diagnóstico; una receta, a una cita y a un medicamento; un médico, a una especialidad. En paciente, email, fecha_fin_vigencia, ocupacion y antecedentes admiten nulos; nombre y ap_paterno son obligatorias.*

> ❗ **Importante**
> Antes de obtener planes, se deben recolectar estadísticas del esquema; de lo contrario, el optimizador recurre a estadísticas dinámicas o a valores por omisión y los planes no son representativos. Ejecutar con un usuario con privilegios de administración en la PDB donde se encuentra el modelo, ajustando `ownname` si es necesario:

```
begin
  dbms_stats.gather_schema_stats (
    ownname => 'CONTROL_MEDICO',
    degree  => 2
  );
end;
/
```

💻 Ejemplo

Obtener el plan de ejecución de la siguiente consulta: mostrar el nombre de los médicos y las fechas de sus citas programadas en el consultorio `C-593`.

```
explain plan
set statement_id = 's1' for
  select m.nombre, c.fecha_cita
  from medico m, cita c
  where m.medico_id = c.medico_id
  and c.consultorio = 'C-593';
```

Para mostrar el plan:

```
set linesize 100
select plan_table_output
from table(dbms_xplan.display('PLAN_TABLE', 's1', 'TYPICAL'));
```

[[keep: plan-01]]

```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------
Plan hash value: 2487834737

-------------------------------------------------------------------------------------------
| Id  | Operation                    | Name       | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |            |    20 |   640 |    50   (0)| 00:00:01 |
|   1 |  NESTED LOOPS                |            |    20 |   640 |    50   (0)| 00:00:01 |
|   2 |   NESTED LOOPS               |            |    20 |   640 |    50   (0)| 00:00:01 |
|*  3 |    TABLE ACCESS FULL         | CITA       |    20 |   420 |    30   (0)| 00:00:01 |
|*  4 |    INDEX UNIQUE SCAN         | MEDICO_PK  |     1 |       |     0   (0)| 00:00:01 |
|   5 |   TABLE ACCESS BY INDEX ROWID| MEDICO     |     1 |    11 |     1   (0)| 00:00:01 |

PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------------------
```

```
-------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   3 - filter("C"."CONSULTORIO"='C-593')
   4 - access("M"."MEDICO_ID"="C"."MEDICO_ID")

Note
-----
   - this is an adaptive plan
```

[[/keep]]

4.3.2. 📖 Lectura de un plan

A nivel básico, un plan en formato `typical` contiene los siguientes elementos.

[[keep: elementos-plan]]

Columna	Significado
`Id`	Identificador de la operación. **No indica el orden de ejecución**; solo sirve para hacer referencia a cada operación. Un `*` a la izquierda indica que la operación tiene predicados, listados en la sección *Predicate Information*.
`Operation`	Tipo de operación: método de acceso, método de join, ordenamiento, agregación, etc. La indentación indica la relación padre-hija.
`Name`	Objeto sobre el que opera: tabla, índice, vista.
`Rows`	Cardinalidad estimada: número de registros que la operación entrega a su padre en una ejecución.
`Bytes`	Volumen estimado de datos que la operación entrega a su padre (registros × tamaño promedio del registro).
`Cost (%CPU)`	Costo estimado acumulado de la operación y sus hijas. El valor entre paréntesis es el porcentaje de ese costo atribuible a procesador (ver la sección *Desglose del costo: I/O y CPU*, al final de los métodos de acceso).
`Time`	Tiempo estimado de la operación, derivado del costo.

[[/keep]]

4.3.2.1. Predicados: access y filter

La sección *Predicate Information* distingue dos formas de aplicar un predicado:

- `access`: el predicado se usa para localizar los registros, por ejemplo, para recorrer solo una parte de un índice. Reduce el trabajo de lectura.
- `filter`: el predicado se aplica después de leer los registros, para descartar los que no cumplen. No reduce lo que se lee, solo lo que se entrega.

En el ejemplo, la operación 3 lee **toda** la tabla `cita` y descarta los registros con un consultorio distinto (`filter`); la operación 4 usa el valor de `medico_id` para ir directamente a la entrada del índice `medico_pk` (`access`).

4.3.2.2. Orden de ejecución: primero la primera hija

¿Qué operación se ejecuta primero?
La regla práctica es **"primero la primera hija, en descenso recursivo"**: cada operación padre invoca a sus hijas en orden, de arriba hacia abajo, y una hija debe producir registros antes de que su padre pueda procesarlos. Para recorrer un plan:

A. Bajar por el árbol desde la operación 0 hasta la primera hoja, siguiendo siempre a la primera hija.
B. Esa hoja produce registros y se los entrega a su padre.
C. El padre invoca a su siguiente hija, si la tiene, y combina ambos resultados.
D. Subir al padre siguiente y repetir hasta llegar a la operación 0.

Aplicado al ejemplo:

A. Operación 3. `table access full` sobre `cita`, filtrando el consultorio `C-593`. Se estiman 20 registros. Es la primera hoja del árbol.
B. Operación 4. Para cada registro de la operación 3, `index unique scan` sobre `medico_pk` con el valor de `c.medico_id`; obtiene a lo más un `rowid`.
C. Operación 2. El primer `nested loops` combina cada cita con el `rowid` del médico correspondiente.
D. Operación 5. Para cada registro de la operación 2, `table access by index rowid` sobre `medico` para obtener la columna `nombre`.
E. Operación 1. El segundo `nested loops` combina ambos resultados.
F. Operación 0. Entrega el resultado al cliente.

Los dos `nested loops` anidados no son dos joins distintos: es la forma en que la base de datos representa un solo join por *nested loops* cuando agrupa (*batching*) los accesos a la tabla interna, primero obteniendo los `rowid` del índice y después leyendo los registros. Los métodos de join se estudian en la parte 2.



Ejercicio en clase 1 - Índice sobre consultorio
Realizar las actividades del siguiente ejercicio. La respuesta no se incluye en estas notas, se sugiere agregar la respuesta en sus apuntes.

Considerar el plan de la consulta de médicos y citas del consultorio `C-593`. Suponer que se crea un índice sobre `cita(consultorio)` y que el optimizador lo usa. ¿Qué operación reemplazaría a la operación 3? ¿Cambiaría la regla de lectura del plan? ¿Qué operación adicional aparecería para obtener `fecha_cita`?

4.3.2.3. La sección Note

Al final del plan pueden aparecer notas relevantes para interpretarlo. Algunos ejemplos son:

- `dynamic statistics used`: el optimizador realizó un muestreo de los datos al optimizar, por falta de estadísticas.
- `this is an adaptive plan`: el plan es adaptativo. Durante la ejecución, un colector de estadísticas compara el número real de registros con el estimado y, si difieren de forma significativa, la base de datos puede cambiar el método de join (por ejemplo, de *nested loops* a *hash join*) sin volver a optimizar. Esta capacidad se introdujo en la versión 12c.

4.3.3. 🔁 Plan real con display_cursor

`explain plan` muestra lo que el optimizador *haría*. Para analizar un problema de desempeño es más útil el plan que *efectivamente se ejecutó*, junto con las cifras reales de cada operación. El procedimiento consta de dos pasos.

❗ **Importante**
Privilegios requeridos. A diferencia de `explain plan`, la función `display_cursor` consulta vistas dinámicas de rendimiento: `v$session` (para ubicar la última sentencia de la sesión), `v$sql`, `v$sql_plan` y `v$sql_plan_statistics_all`. Un usuario sin privilegios de administración debe recibir permiso de lectura sobre ellas. Las vistas `v$` son sinónimos públicos de las vistas `v_$`, por lo que el privilegio se otorga sobre estas últimas; intentar otorgarlo sobre `v$sql_plan_statistics_all` produce el error `ORA-02030: can only select from fixed tables/views`. El script de creación del usuario del caso de estudio ya incluye estas instrucciones:

```
grant select on v_$session to control_medico;
grant select on v_$sql to control_medico;
grant select on v_$sql_plan to control_medico;
grant select on v_$sql_plan_statistics_all to control_medico;
```

Paso 1. Ejecutar la consulta con el hint `gather_plan_statistics`, que indica a la base de datos que recolecte estadísticas de ejecución por operación:

```
set linesize 100
set pagesize 100
select /*+ gather_plan_statistics */ m.nombre, c.fecha_cita
from medico m, cita c
where m.medico_id = c.medico_id
and c.consultorio = 'C-593';
```

Paso 2. Inmediatamente después, en la misma sesión, mostrar el plan de la última sentencia ejecutada con el formato `allstats last`:

```
select *
from table(dbms_xplan.display_cursor(format => 'ALLSTATS LAST'));
```

El plan real agrega, entre otras, las siguientes columnas:

[[keep: elementos-plan-real]]

Columna	Significado
`Starts`	Número de veces que se ejecutó la operación. En un *nested loops*, la tabla interna se ejecuta una vez por cada registro de la externa.
`E-Rows`	Registros **estimados** por el optimizador (equivale a `Rows`).
`A-Rows`	Registros **reales** producidos por la operación.
`A-Time`	Tiempo real acumulado.
`Buffers`	Bloques leídos de memoria (lecturas lógicas).

[[/keep]]

La comparación entre `E-Rows` y `A-Rows` es la herramienta de diagnóstico más importante: cuando la estimación difiere en órdenes de magnitud del valor real, el optimizador eligió el plan con información incorrecta, y casi siempre la causa está en las estadísticas. Agregando `+adaptive` al formato se muestran también las operaciones alternativas de un plan adaptativo, marcando las que quedaron inactivas.

[[keep: plan-02]]

```
PLAN_TABLE_OUTPUT
------------------------------------------------------------------------------------------------------
SQL_ID  fu4uqsv3tsvj8, child number 0
-------------------------------------
select /*+ gather_plan_statistics */ m.nombre, c.fecha_cita from medico
m, cita c where m.medico_id = c.medico_id and c.consultorio = 'C-593'

Plan hash value: 2487834737

------------------------------------------------------------------------------------------------------
| Id  | Operation                    | Name      | Starts | E-Rows | A-Rows |   A-Time   | Buffers |
------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |           |      1 |        |     20 |00:00:00.01 |     151 |
|   1 |  NESTED LOOPS                |           |      1 |     20 |     20 |00:00:00.01 |     151 |
|   2 |   NESTED LOOPS               |           |      1 |     20 |     20 |00:00:00.01 |     131 |
|*  3 |    TABLE ACCESS FULL         | CITA      |      1 |     20 |     20 |00:00:00.01 |     108 |
|*  4 |    INDEX UNIQUE SCAN         | MEDICO_PK |     20 |      1 |     20 |00:00:00.01 |      23 |
|   5 |   TABLE ACCESS BY INDEX ROWID| MEDICO    |     20 |      1 |     20 |00:00:00.01 |      20 |
------------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------
```

```
   3 - filter("C"."CONSULTORIO"='C-593')
   4 - access("M"."MEDICO_ID"="C"."MEDICO_ID")

Note
-----
   - this is an adaptive plan
```

‖/keep‖

📝 **Nota**
🌐 **Modernización: SQL Analysis Report (23ai).** A partir de la versión 23ai, el plan que muestra `dbms_xplan` puede incluir al final una sección llamada *SQL Analysis Report*, en la que el optimizador señala construcciones de la sentencia que impiden un buen plan; por ejemplo, una función aplicada a una columna indexada en el `where`, o condiciones de join faltantes que provocan un producto cartesiano. La misma información aparece en el reporte de *SQL Monitor*. Es un apoyo didáctico útil: varios de los casos de "¿por qué no se usa el índice?" de la siguiente sección los detecta el propio optimizador.

📝 **Nota**
🌐 **Modernización: el mismo concepto en otros motores.** PostgreSQL ofrece `explain` (plan estimado) y `explain analyze` (ejecuta la sentencia y reporta registros y tiempos reales por nodo), con la misma lógica de comparar estimación contra realidad. Sus costos también son unidades arbitrarias, calibradas respecto a la lectura secuencial de una página (`seq_page_cost`). En los motores distribuidos el plan agrega operaciones de intercambio de datos entre nodos: Oracle muestra operaciones `REMOTE` con el SQL enviado a cada sitio, y Spanner, como se vio en el Tema 3, muestra operadores como *Distributed Union*. La lectura de planes distribuidos se estudia en la parte 4.

4.4. 🧭 Métodos de acceso a datos (access paths)

¿Por qué una misma tabla, con el mismo índice, a veces se lee completa y a veces no?
Un **método de acceso** es la técnica con la que una consulta obtiene los registros de una fuente de datos. Es el concepto fundamental para entender y generar planes eficientes. A nivel básico existen dos familias: **acceso a tablas** y **acceso a índices**. La siguiente tabla resume los métodos de acceso de la base de datos según la estructura sobre la que operan.

Método de acceso	Tablas	Índices B-tree	Índices bitmap	Clústeres
Full table scan	X			
Table access by rowid	X			
Sample table scan	X			
In-memory table scan	X			
Index unique scan		X		

Método de acceso	Tablas	Índices B-tree	Índices bitmap	Clústeres
Index range scan		X		
Index full scan		X		
Index fast full scan		X		
Index skip scan		X		
Index join scan		X		
Bitmap index single value			X	
Bitmap index range scan			X	
Bitmap merge			X	
Cluster scan				X
Hash scan				X

Un **clúster de tablas** es un conjunto de tablas que comparten columnas en común y cuyos registros relacionados se almacenan en los mismos bloques de datos; esto permite leer un solo bloque para obtener datos de varias tablas. Los métodos *bitmap* corresponden a índices de tipo bitmap y *hash scan* al uso de clústeres hash. Para efectos del curso se revisan únicamente los métodos de acceso a tablas y a índices B-tree.
La forma en que una tabla organiza el almacenamiento de sus registros también influye en la selección del método de acceso:

- Heap-organized table (por omisión). Los registros no se almacenan en algún orden en particular: cada registro se almacena en el primer espacio disponible del segmento de datos. El término *heap* (montón) alude justamente a esa falta de orden: los registros se van "amontonando" donde haya espacio. No tiene relación con la estructura de datos *heap* (montículo) empleada, por ejemplo, en colas de prioridad.
- Index-organized table (IOT). Los registros se almacenan en la estructura de un índice B-tree, ordenados por el valor de la llave primaria.
- External table. Tabla de solo lectura cuyos metadatos se almacenan en la base de datos y cuyos datos residen en archivos externos.

4.4.1. 📑 El índice como fuente de rowids

Para efectos de un plan de ejecución, un índice B-tree puede visualizarse como una tabla de dos columnas: la **llave** (el valor indexado) y el `rowid`. Por ejemplo, si la columna `email` de la tabla `paciente` está indexada, el índice contiene:

rowid	email
01	`hugo@mail.com`
02	`juan@mail.com`
03	`paco@mail.com`

rowid	email
...	...

Un `rowid` es la dirección física de un registro (archivo de datos, bloque y posición dentro del bloque) y representa la forma más rápida de localizarlo. Por simplicidad se emplean los valores 01, 02, 03; en la base de datos un `rowid` se representa con 18 caracteres. Las llaves de un índice B-tree se almacenan **ordenadas**. El objetivo de leer un índice es obtener una lista de `rowid` con la cual acceder eficientemente a los registros de la tabla; si el índice contiene todas las columnas que la consulta necesita, ni siquiera es necesario visitar la tabla.
Un detalle con consecuencias directas en los planes es el manejo de nulos:

- Un índice B-tree no almacena entradas cuya llave es completamente nula. En un índice de una sola columna, los registros con esa columna en `null` no aparecen en el índice.
- En un índice compuesto, una entrada se almacena si al menos una de sus columnas no es nula.

Por lo tanto, si una columna indexada admite nulos y la consulta necesita **todos** los registros, el optimizador no puede resolverla solo con el índice, porque le faltarían los registros nulos. Si la columna está declarada `not null`, o si el predicado excluye los nulos (`is not null`, una igualdad, un rango), el índice sí es suficiente.
Por lo anterior, una técnica común para indexar nulos es agregar al índice una columna `not null` o una constante, por ejemplo `create index ... on paciente(email, 0)`.

4.4.2. 🗂️ Acceso a tablas

Generalmente, acceder a un índice requiere menos recursos que acceder a la tabla, porque su estructura es mucho más pequeña. Sin embargo, hay casos en los que el acceso directo a la tabla es la mejor opción, como se muestra a continuación.

4.4.2.1. Table access full

En un *full table scan*, la base de datos lee **todos** los bloques con formato del segmento de la tabla, hasta la marca de agua (*high water mark*, HWM). Los bloques sin formato, posteriores a la marca de agua, no se leen. Cada bloque se lee una sola vez y la lectura es **multibloque**: en cada operación de I/O se lee un conjunto de bloques contiguos, cuyo tamaño máximo define el parámetro `db_file_multiblock_read_count`.

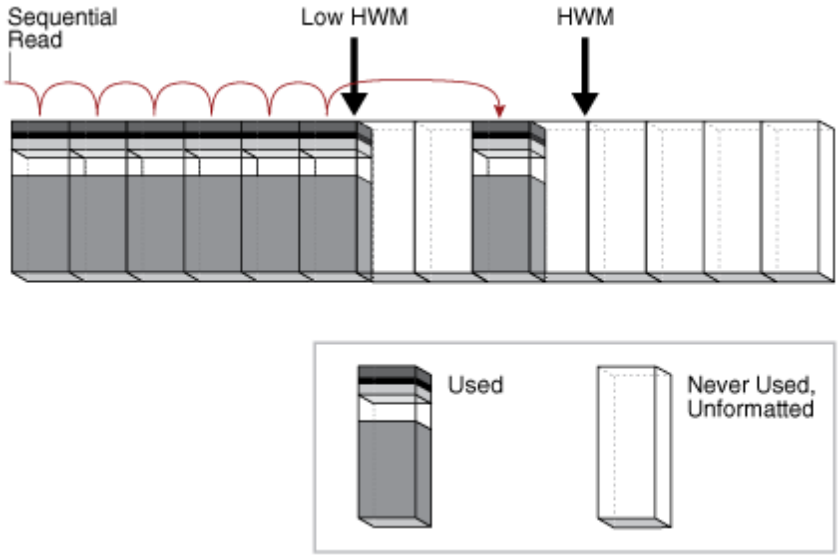


Figura 05. *Full table scan: la lectura secuencial multibloque recorre los bloques del segmento desde el inicio hasta la marca de agua alta (HWM). Antes de la marca baja (low HWM) todos los bloques están usados; entre ambas solo se leen los bloques con formato, y los posteriores a la HWM, nunca usados, no se leen.*

En general, el optimizador selecciona este método en las siguientes situaciones:

- No existe un índice aplicable a los predicados de la consulta.
- El predicado aplica una función a la columna indexada y no existe un índice basado en dicha función. Por ejemplo, `where upper(email) = ...` no puede usar un índice sobre `email`.
- La consulta requiere registros con nulos en una columna indexada que los admite (ver la sección anterior).
- La selectividad es baja: la consulta obtiene una fracción grande de la tabla. Leer muchos bloques en pocas operaciones multibloque es más eficiente que leer bloques individuales con alta frecuencia a partir de cada `rowid`.
- Las estadísticas están desactualizadas. Por ejemplo, la tabla era pequeña cuando se recolectaron sus estadísticas y después creció; el optimizador sigue creyendo que es pequeña y no detecta que el índice ya es más conveniente.
- La tabla es pequeña: leerla completa con pocas lecturas multibloque cuesta menos que recorrer un índice y después visitar la tabla.
- La consulta usa un hint que fuerza el *full table scan*: `full(alias)`.

Lectura multibloque y el parámetro `db_file_multiblock_read_count`. Este parámetro indica el número máximo de bloques contiguos que la base de datos solicita al sistema operativo en una sola operación de I/O durante un recorrido completo, ya sea un *full table scan* o un *index fast full scan*. Así, en lugar de leer la tabla bloque por bloque, se leen grupos de bloques: una tabla de 1 024 bloques puede leerse con 8 operaciones de 128 bloques en lugar de 1 024 operaciones de un bloque.

```
show parameter db_file_multiblock_read_count
```

```
NAME                                 TYPE        VALUE
------------------------------------ ----------- -----
db_file_multiblock_read_count        integer     128
```

Algunas precisiones sobre su funcionamiento:

- Es un máximo. Una lectura multibloque no cruza los límites de un *extent* ni incluye bloques que ya se encuentran en el *buffer cache*, por lo que muchas lecturas traen menos bloques que el valor configurado. El tamaño de los *extents*, a su vez, no es fijo: en un tablespace localmente administrado con asignación automática (`autoallocate`, el valor por omisión), los *extents* crecen conforme crece el segmento. Con bloques de 8 KB:

Extents del segmento	Tamaño de cada extent	Bloques por extent
Primeros 16 (hasta 1 MB)	64 KB	8
Siguientes 63 (hasta 64 MB)	1 MB	128
Siguientes 120 (hasta 1 GB)	8 MB	1 024
Posteriores	64 MB	8 192

  Así, solo el primer megabyte de una tabla se lee en grupos de 8 bloques; a partir de ahí, los *extents* ya admiten lecturas de 128 bloques (1 MB), el valor típico del parámetro. Por ejemplo, la tabla `paciente`, de aproximadamente 1 756 bloques (14 MB), se leería con 16 lecturas de 8 bloques y alrededor de 13 lecturas de 128. En un tablespace con `uniform size`, todos los *extents* tienen el mismo tamaño, y si este es menor que 1 MB limita las lecturas multibloque en todo el segmento. El optimizador no considera los *extents* al estimar el costo: el límite afecta la ejecución real, no el costo calculado.
- Los accesos por índice no lo emplean. Un *index unique scan*, un *index range scan* o un *table access by rowid* leen un bloque a la vez: raíz, rama, hoja y, después, el bloque de la tabla. En las estadísticas de espera de la base de datos, esta diferencia se observa como eventos `db file sequential read` (un bloque) frente a `db file scattered read` (multibloque).
- Ajuste automático. Si el parámetro no se configura explícitamente, la base de datos ajusta el tamaño real de la lectura de forma automática, típicamente a 1 MB (128 bloques de 8 KB). Para estimar costos, el optimizador emplea un valor más conservador, tomado de las estadísticas del sistema si fueron recolectadas.

Su utilidad real es, por tanto, **abaratar los recorridos completos**: mientras más bloques se leen por operación, menos operaciones de I/O requiere un *full scan* y menor es su costo estimado.

¿Índice o *full table scan*? Comparación de costos. No existe un umbral fijo, ni de bloques ni de porcentaje de registros, a partir del cual se elija un método u otro. El optimizador estima el costo de cada alternativa y elige la de menor costo:

[[keep: formula-01]]

- Full table scan. Su costo prácticamente no depende de cuántos registros se soliciten, porque siempre lee todos los bloques hasta la HWM. Su costo de I/O es aproximadamente proporcional a $bloques / MBRC$, donde *MBRC* es el número de bloques por lectura multibloque.
- Acceso por índice. Su costo crece con la selectividad, porque cada registro obtenido implica visitar bloques del índice y, después, bloques de la tabla, uno a la vez. De forma aproximada:

[[/keep]]
[[keep: formula-02]]

$$costo_{indice} \approx blevel + hojas * sel + factor_clusterizacion * sel$$

[[/keep]]
donde:

- *blevel*: es el número de niveles de ramas del índice
- *hojas*: el número de bloques hoja,
- *sel*: la selectividad del predicado
- *factor de clusterización*: mide qué tan ordenados están los registros de la tabla con respecto al índice (cuántos bloques distintos de la tabla hay que visitar al recorrer el índice en orden).

El punto de equilibrio depende, entonces, de la selectividad, del tamaño de la tabla, del MBRC y, sobre todo, del factor de clusterización. Con registros bien agrupados, un índice puede convenir aún cuando se obtiene una fracción considerable de la tabla; con registros dispersos, el *full table scan* puede ser más barato aunque se obtenga menos del 1 %. Por ello, reglas prácticas del tipo "si se obtiene más del 10 % de los registros conviene el *full table scan*" son solo aproximaciones. Por la misma razón, en una **tabla pequeña** el *full table scan* suele ganar: unas cuantas lecturas multibloque cuestan menos que recorrer el índice y visitar después la tabla.

📝 **Nota**
🔗 El factor de clusterización y las demás estadísticas de un índice (*blevel*, número de hojas, llaves distintas) se estudian en la parte 3.

💻 **Ejemplo**

```
explain plan for
  select * from paciente;

select plan_table_output
from table(dbms_xplan.display);
```

```
Plan hash value: 588422155

-------------------------------------------------------------------------------
| Id  | Operation         | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |          | 15302 |   26M |   477   (0)| 00:00:01 |
|   1 |  TABLE ACCESS FULL| PACIENTE | 15302 |   26M |   477   (0)| 00:00:01 |
-------------------------------------------------------------------------------
```

4.4.2.2. Table access by rowid

- Es la forma más rápida de acceder a un registro: el `rowid` indica exactamente el bloque y la posición donde se encuentra.
- Generalmente se emplea después de recorrer un índice, que proporciona los `rowid`.
- Si el índice contiene todas las columnas requeridas por la consulta, este acceso se omite.

💻 Ejemplo

```
explain plan for
  select nombre from paciente where paciente_id = 3;

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-03]]

```
Plan hash value: 3341724136

-----------------------------------------------------------------------------------------
| Id  | Operation                    | Name        | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |             |     1 |    35 |     2   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID| PACIENTE    |     1 |    35 |     2   (0)| 00:00:01 |
|*  2 |   INDEX UNIQUE SCAN          | PACIENTE_PK |     1 |       |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("PACIENTE_ID"=3)
```

[[/keep]]

Cuando el índice proporciona varios `rowid`, la base de datos puede ordenarlos por bloque antes de visitar la tabla, para leer cada bloque una sola vez aunque contenga varios de los registros buscados. Esta variante se indica con la palabra `BATCHED`:

```
explain plan for
  select nombre from paciente where paciente_id > 18500;

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-04]]

```
Plan hash value: 2503382986

-----------------------------------------------------------------------------------------------
| Id  | Operation                           | Name        | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                    |             |     1 |    35 |     1   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID BATCHED| PACIENTE    |     1 |    35 |     1   (0)| 00:00:01 |
|*  2 |   INDEX RANGE SCAN                  | PACIENTE_PK |     1 |       |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("PACIENTE_ID">18500)
```

[[/keep]]

4.4.2.3. Sample table scan

- Obtiene una muestra aleatoria de los registros de una tabla, o de una sentencia `select` compleja con joins y vistas.
- La muestra se expresa como un porcentaje en el rango [0.000001, 100).
- Con `sample (p)` el porcentaje se aplica a los registros; con `sample block (p)` se aplica a los bloques, lo cual es más rápido pero menos uniforme si los datos están agrupados físicamente. En ambos casos el resultado es aproximado.

💻 Ejemplo

```
explain plan for
  select * from cita sample block (10);

select plan_table_output from table(dbms_xplan.display);
```

```
Plan hash value: 1426444112

-------------------------------------------------------------------------------
| Id  | Operation           | Name | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT    |      |   208 | 16848 |     5   (0)| 00:00:01 |
|   1 |  TABLE ACCESS SAMPLE| CITA |   208 | 16848 |     5   (0)| 00:00:01 |
-------------------------------------------------------------------------------
```

4.4.2.4. In-memory table scan

- Obtiene los registros de una tabla cuyas columnas están cargadas en memoria en formato columnar, en el área llamada *IM column store*.
- El *IM column store* es un área opcional de la SGA que almacena copias de tablas, particiones o columnas en un formato columnar comprimido, que agiliza el escaneo y el filtrado de grandes volúmenes de datos. Los datos permanecen también en su formato de registros tradicional en disco y en el *buffer cache*.
- Solo se menciona de forma conceptual: su configuración y práctica corresponden a la asignatura de BDA.

💻 Ejemplo

Plan real obtenido con `display_cursor`:

[[keep: plan-05]]

```
SQL_ID  2mb4h57x8pabw, child number 0
-------------------------------------
select * from oe.product_information where list_price > 10 order by product_id

Plan hash value: 2256295385

-----------------------------------------------------------------------------------------------------------
| Id | Operation                   | Name                | Rows | Bytes |TempSpc| Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------
|  0 | SELECT STATEMENT            |                     |      |       |       |    21 (100)|          |
|  1 |  SORT ORDER BY              |                     |  285 | 62415 | 82000 |    21   (5)| 00:00:01 |
|* 2 |   TABLE ACCESS INMEMORY FULL| PRODUCT_INFORMATION |  285 | 62415 |       |     5   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - inmemory("LIST_PRICE">10)
       filter("LIST_PRICE">10)
```

[[/keep]]

Observar que el predicado aparece como `inmemory`: el filtro se evalúa directamente sobre el formato columnar, sin reconstruir registros completos.

4.4.3. 🌲 Acceso a índices B-tree

Para efectos del curso, solo se consideran los índices de tipo B-tree. Las figuras de esta sección muestran su estructura: los **bloques rama** (*branch blocks*) guían la búsqueda, y los **bloques hoja** (*leaf blocks*) contienen las parejas llave–`rowid` ordenadas y están enlazados entre sí, lo que permite recorrerlos horizontalmente en ambos sentidos.

4.4.3.1. Index unique scan

- Regresa a lo más un `rowid`.
- Se emplea cuando el predicado aplica un operador de igualdad sobre todas las columnas de un índice `unique` (incluidos los índices de llave primaria).
- El índice se recorre desde la raíz hasta la hoja que contiene la llave; al tratarse de un índice único, no es necesario continuar el recorrido una vez encontrada.

La siguiente figura muestra el recorrido para encontrar el `rowid` de la llave 19.

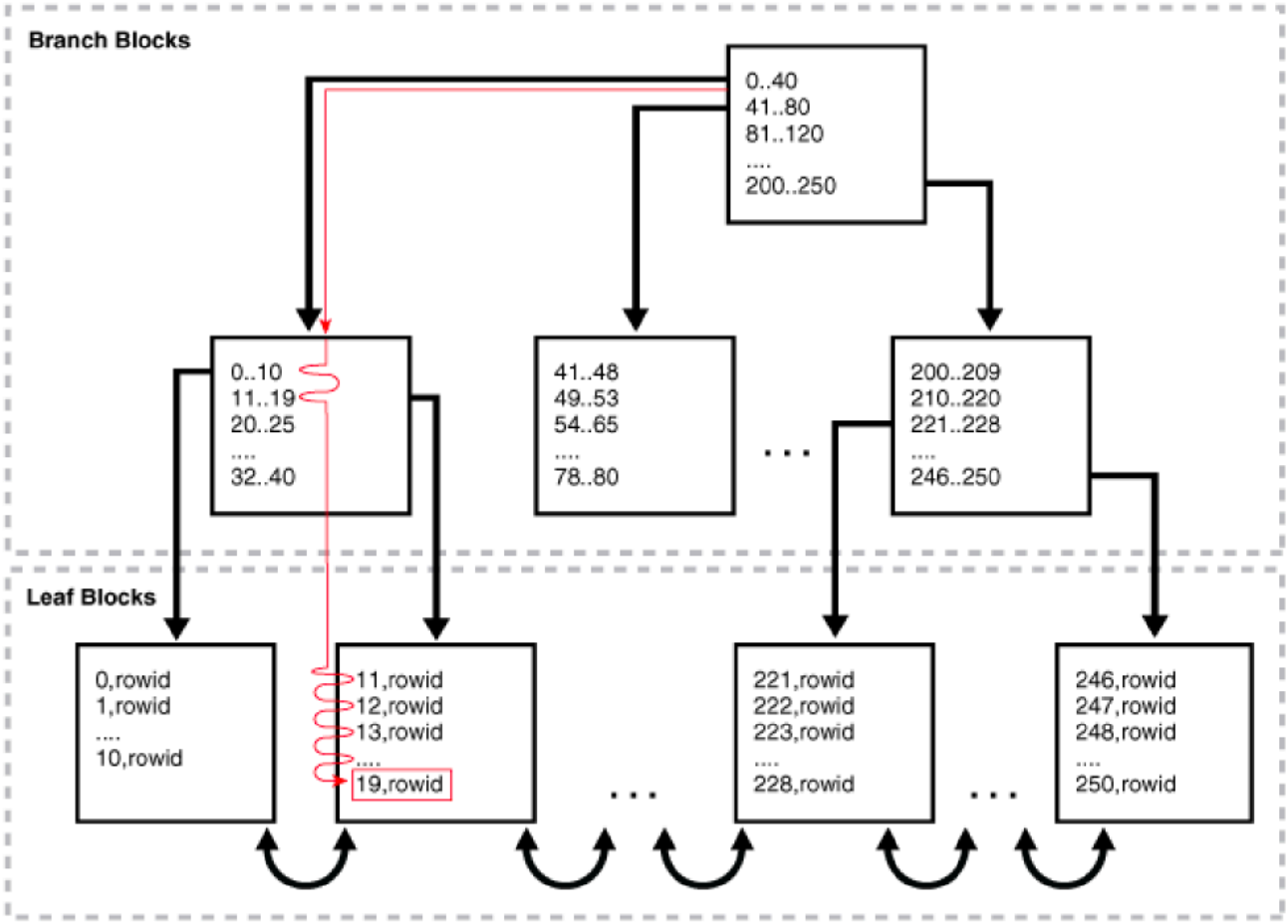


Figura 06. *Index unique scan en un B-tree: desde la raíz (rangos 0..40 a 200..250), la búsqueda baja a la rama 0..40 y de ahí a la hoja con las llaves 11 a 19, donde encuentra 19,rowid y se detiene. Las hojas contienen parejas llave,rowid ordenadas y enlazadas en ambos sentidos.*

💻 Ejemplo

```
create unique index paciente_email_idx on paciente(email);

explain plan for
  select email from paciente where email = 'jonh@mail.com';

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-06]]

```
Plan hash value: 839293805

-----------------------------------------------------------------------------------------
| Id  | Operation         | Name               | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |                    |     1 |   102 |     1   (0)| 00:00:01 |
|*  1 |  INDEX UNIQUE SCAN| PACIENTE_EMAIL_IDX |     1 |   102 |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("EMAIL"='jonh@mail.com')
```

[[/keep]]

Observar que no aparece `table access by index rowid`: la consulta solo solicita `email`, que ya está en el índice.



Ejercicio en clase 2 - Índice sobre una columna con nulos

Realizar las actividades del siguiente ejercicio. La respuesta no se incluye en estas notas, se sugiere agregar la respuesta en sus apuntes.

Considerar el índice `paciente_email_idx` y el siguiente plan. ¿Por qué el optimizador no emplea el índice, si la consulta solo solicita la columna indexada? Proponer dos formas de lograr que la consulta se resuelva leyendo únicamente el índice, indicando cuál de ellas modifica el resultado.

```
explain plan for
  select email from paciente;

select plan_table_output from table(dbms_xplan.display);
```

```
Plan hash value: 588422155

-------------------------------------------------------------------------------
| Id  | Operation         | Name     | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT  |          | 16584 |  1651K|   477   (0)| 00:00:01 |
|   1 |  TABLE ACCESS FULL| PACIENTE | 16584 |  1651K|   477   (0)| 00:00:01 |
-------------------------------------------------------------------------------
```

4.4.3.2. Index range scan

- Recorre de forma ordenada un rango de llaves del índice.
- Por omisión, las llaves se almacenan en orden ascendente y se recorren en ese mismo orden; si la consulta requiere orden descendente, el recorrido puede hacerse en sentido inverso (`index range scan descending`), evitando un ordenamiento.
- Se emplea, entre otros casos, cuando:
    - El predicado aplica operadores de rango sobre la columna indexada: `>`, `>=`, `<`, `<=`, `between`, o `like` con un prefijo fijo (`like 'bob@%'`).
    - El índice no es único y se aplica una igualdad, por lo que pueden obtenerse varios `rowid`.
    - El predicado aplica igualdad solo sobre las columnas iniciales de un índice compuesto.

En la siguiente figura, el recorrido desciende hasta la primera hoja que contiene la llave 20 y, a partir de ahí, avanza horizontalmente por los punteros entre hojas hasta la última llave del rango (por ejemplo, 40).

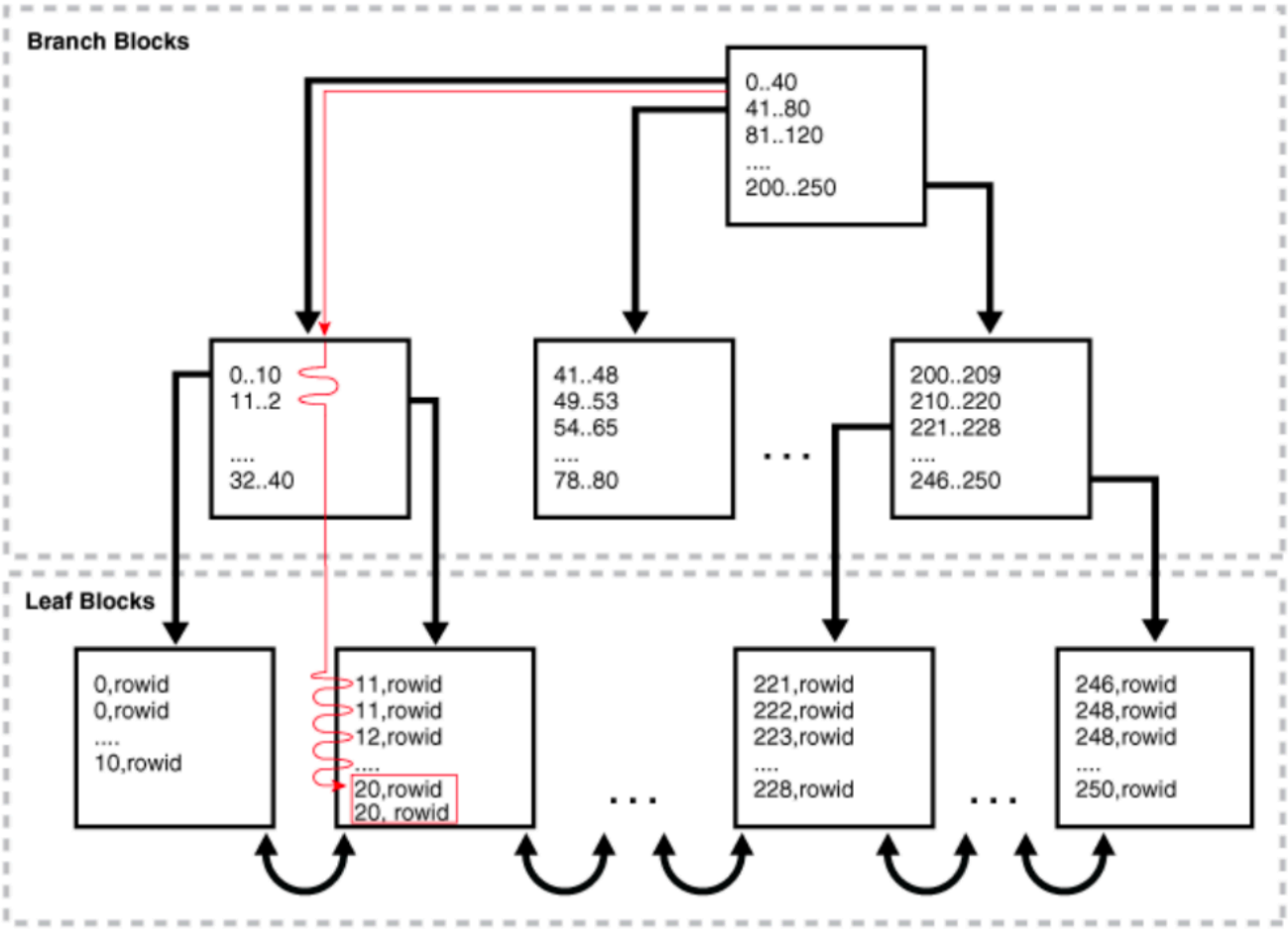


Figura 07. *Index range scan en un B-tree: la búsqueda baja desde la raíz por la rama 0..40 hasta la hoja con la primera llave del rango (20) y avanza horizontalmente por los enlaces entre hojas hasta la última llave del rango. Las llaves repetidas (dos entradas con 20) son propias de un índice no único.*

💻 Ejemplo

```
explain plan for
  select email from paciente where email like 'bob@%' order by email desc;

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-07]]

```
Plan hash value: 3471813995

-----------------------------------------------------------------------------------------------
| Id  | Operation                    | Name               | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT             |                    |     1 |   102 |     1   (0)| 00:00:01 |
|*  1 |  INDEX RANGE SCAN DESCENDING| PACIENTE_EMAIL_IDX |     1 |   102 |     1   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - access("EMAIL" LIKE 'bob@%')
       filter("EMAIL" LIKE 'bob@%')
```

[[/keep]]

Observar que no aparece una operación `sort order by`: el índice entrega las llaves ya ordenadas, en sentido descendente.



Ejercicio en clase 3 - Ordenamiento por la llave primaria

Realizar las actividades del siguiente ejercicio. La respuesta no se incluye en estas notas, se sugiere agregar la respuesta en sus apuntes.

Analizar la siguiente sentencia y su plan. Existen el índice `paciente_nombre_idx` y el índice de la llave primaria `paciente_pk`. ¿Por qué el optimizador no emplea ninguno de los dos? ¿Qué operaciones serían necesarias si usara `paciente_pk` para evitar el ordenamiento, y por qué resultarían más costosas?

```
create index paciente_nombre_idx on paciente(nombre);

explain plan for
  select nombre from paciente order by paciente_id desc;

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-08]]

```
Plan hash value: 375339519

-------------------------------------------------------------------------------------
| Id  | Operation          | Name     | Rows  | Bytes |TempSpc| Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |          | 16584 |   566K|       |   635   (1)| 00:00:01 |
|   1 |  SORT ORDER BY     |          | 16584 |   566K|   728K|   635   (1)| 00:00:01 |
|   2 |   TABLE ACCESS FULL| PACIENTE | 16584 |   566K|       |   477   (0)| 00:00:01 |
-------------------------------------------------------------------------------------
```

[[/keep]]

4.4.3.3. Index full scan

- Lee el índice completo y en orden, recorriendo las hojas una por una. Como las llaves están ordenadas, puede evitar una operación de ordenamiento.
- Se emplea, entre otros casos, cuando:
  - La sentencia incluye `order by` sobre las columnas del índice y todas las columnas solicitadas están en el índice.
  - No existe predicado, pero todas las columnas solicitadas están en el índice y al menos una de las columnas indexadas está declarada `not null` (lo cual garantiza que el índice contiene todos los registros).
  - El predicado hace referencia a una columna del índice que no necesariamente es la primera de un índice compuesto.

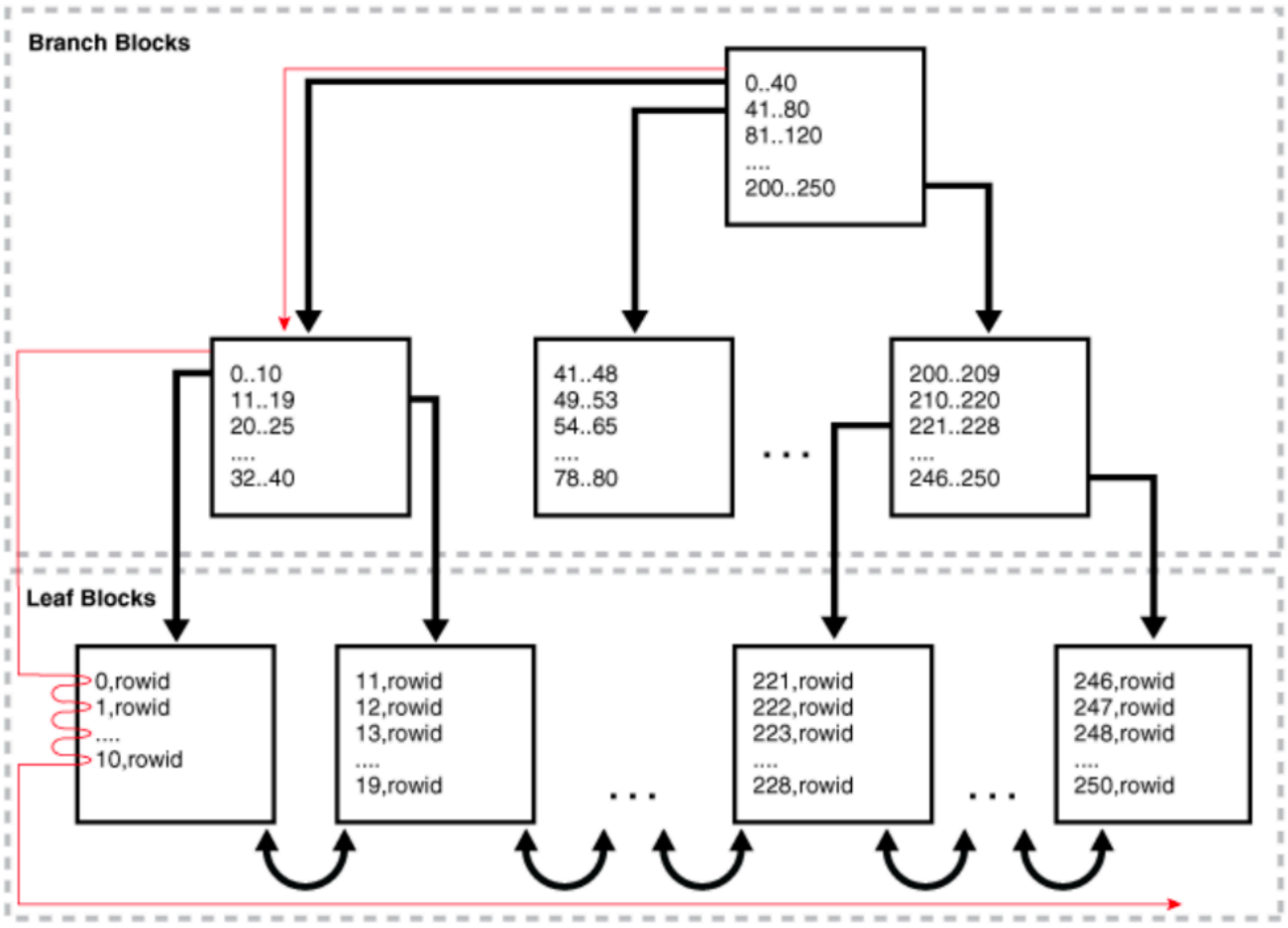


Figura 08. *Index full scan en un B-tree: la búsqueda baja desde la raíz hasta la primera hoja (llaves 0 a 10) y lee todas las hojas en orden, de izquierda a derecha, hasta la última (246 a 250). Las llaves salen ordenadas, por lo que puede evitarse un ordenamiento posterior.*

💻 Ejemplo

La columna `nombre` está declarada `not null`, por lo que el índice `paciente_nombre_idx` contiene a todos los pacientes:

```
explain plan for
  select nombre from paciente order by nombre;

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-09]]

```
Plan hash value: 898347700

-------------------------------------------------------------------------------------------
| Id  | Operation        | Name                | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT |                     | 16584 |   356K|    45   (0)| 00:00:01 |
|   1 |  INDEX FULL SCAN | PACIENTE_NOMBRE_IDX | 16584 |   356K|    45   (0)| 00:00:01 |
-------------------------------------------------------------------------------------------
```

[[/keep]]
Comparar con el ejercicio en clase 3: aquí el índice proporciona tanto los datos como el orden, por lo que el costo baja de 635 a 45.

4.4.3.4. Index fast full scan

- Lee todos los bloques del índice tal como están almacenados en disco, sin respetar el orden de las llaves y con lecturas multibloque, de forma similar a un *full table scan* pero sobre una estructura más pequeña. Puede ejecutarse en paralelo.
- Físicamente, los bloques del índice no necesariamente están ordenados dentro de su segmento, por lo que el resultado no sale ordenado.
- Se emplea cuando la consulta solicita únicamente columnas contenidas en el índice y no se requiere orden; el índice funciona como una "tabla reducida".
- Igual que en el *index full scan*, el índice debe contener todos los registros necesarios: alguna columna indexada `not null` o un predicado que excluya los nulos.

💻 Ejemplo

Este plan responde a la segunda mitad del ejercicio en clase 2:

```
explain plan for
  select email from paciente where email is not null;

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-10]]

```
Plan hash value: 3687434030

-------------------------------------------------------------------------------------------
| Id  | Operation            | Name               | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT     |                    | 14438 |  1438K|    20   (0)| 00:00:01 |
|*  1 |  INDEX FAST FULL SCAN| PACIENTE_EMAIL_IDX | 14438 |  1438K|    20   (0)| 00:00:01 |
-------------------------------------------------------------------------------------------
```

[[/keep]]

💻 Ejemplo

Un conteo de registros puede resolverse leyendo el índice de la llave primaria, que contiene a todos los registros y es más pequeño que la tabla:

```
explain plan for
  select count(*) from paciente;

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-11]]

```
Plan hash value: 2895057998

-----------------------------------------------------------------------------
| Id  | Operation             | Name        | Rows  | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------
|   0 | SELECT STATEMENT      |             |     1 |     9   (0)| 00:00:01 |
|   1 |  SORT AGGREGATE       |             |     1 |            |          |
|   2 |   INDEX FAST FULL SCAN| PACIENTE_PK | 15000 |     9   (0)| 00:00:01 |
-----------------------------------------------------------------------------
```

[[/keep]]

4.4.3.5. Index skip scan

Se emplea con índices compuestos cuando:

- La consulta no incluye en el predicado la primera columna del índice.
- La primera columna tiene muy pocos valores distintos.
- La segunda columna tiene una gran cantidad de valores distintos.

💻 Ejemplo

Suponer un índice compuesto sobre las columnas `genero` y `email` de la tabla `paciente`. Sus entradas se verían así:

```
. . .
F,wolf@company.example.com,rowid
F,wolsey@company.example.com,rowid
F,wood@company.example.com,rowid
F,yang@company.example.com,rowid
F,zimmerman@company.example.com,rowid
M,abbassi@company.example.com,rowid
M,abbey@company.example.com,rowid
. . .
```

Para buscar un correo sin conocer el género, la base de datos divide lógicamente el índice en **subíndices**, uno por cada valor distinto de la primera columna, y "salta" de uno a otro, realizando en cada uno una búsqueda por la segunda columna. El efecto es equivalente a la siguiente consulta:

```
select * from paciente
where genero = 'F' and email = 'abbey@company.example.com'
union all
select * from paciente
where genero = 'M' and email = 'abbey@company.example.com';
```

Mientras menos valores distintos tenga la primera columna, menos subíndices hay que recorrer y más conveniente resulta el método.

```
create index paciente_genero_email_idx on paciente(genero, email);

explain plan for
  select * from paciente where email = 'smith@mail.com';

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-12]]

```
Plan hash value: 3560863078

-----------------------------------------------------------------------------------------------------------------
| Id  | Operation                           | Name                       | Rows  | Bytes | Cost (%CPU)| Time     |
-----------------------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT                    |                            |     1 |   773 |     4   (0)| 00:00:01 |
|   1 |  TABLE ACCESS BY INDEX ROWID BATCHED| PACIENTE                   |     1 |   773 |     4   (0)| 00:00:01 |
|*  2 |   INDEX SKIP SCAN                   | PACIENTE_GENERO_EMAIL_IDX |     1 |       |     3   (0)| 00:00:01 |
-----------------------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   2 - access("EMAIL"='smith@mail.com')
       filter("EMAIL"='smith@mail.com')
```

[[/keep]]

4.4.3.6. Index join scan

- Consiste en realizar un hash join entre varios índices de la misma tabla, unidos por el `rowid`, cuando entre todos contienen las columnas que la consulta solicita.
- De esta forma, la consulta se resuelve sin acceder a la tabla: se hace join entre índices.

💻 Ejemplo

```
create index paciente_nombre_idx on paciente(nombre);
create index paciente_ap_paterno_idx on paciente(ap_paterno);

explain plan for
  select nombre, ap_paterno from paciente where nombre like 'A%';

select plan_table_output from table(dbms_xplan.display);
```

[[keep: plan-13]]

```
Plan hash value: 3337046678

-------------------------------------------------------------------------------------------------
| Id  | Operation                | Name                     | Rows  | Bytes | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------------------------
|   0 | SELECT STATEMENT         |                          |   422 |  6330 |    42   (0)| 00:00:01 |
|*  1 |  VIEW                    | index$_join$_001         |   422 |  6330 |    42   (0)| 00:00:01 |
|*  2 |   HASH JOIN              |                          |       |       |            |          |
|*  3 |    INDEX RANGE SCAN      | PACIENTE_NOMBRE_IDX      |   422 |  6330 |     3   (0)| 00:00:01 |
|   4 |    INDEX FAST FULL SCAN  | PACIENTE_AP_PATERNO_IDX  |   422 |  6330 |    49   (0)| 00:00:01 |
-------------------------------------------------------------------------------------------------

Predicate Information (identified by operation id):
---------------------------------------------------

   1 - filter("NOMBRE" LIKE 'A%')
   2 - access(ROWID=ROWID)
   3 - access("NOMBRE" LIKE 'A%')
```

[[/keep]]

Lectura del plan, aplicando la regla de la primera hija:

A. Operación 3. `index range scan` sobre `paciente_nombre_idx` para obtener los nombres que inician con A y sus `rowid`.
B. Operación 4. `index fast full scan` sobre `paciente_ap_paterno_idx` para obtener todos los apellidos con sus `rowid`.
C. Operación 2. `hash join` entre ambos resultados, usando el `rowid` como llave (`access(ROWID=ROWID)`).
D. Operación 1. El resultado se presenta como una vista interna (`index$_join$_001`), sobre la cual se vuelve a verificar el predicado.



Ejercicio en clase 4 - Selección del método de acceso

Realizar las actividades del siguiente ejercicio. La respuesta no se incluye en estas notas, se sugiere agregar la respuesta en sus apuntes.

Para cada una de las siguientes consultas sobre `paciente`, indicar el método de acceso más probable y justificarlo. Considerar los índices `paciente_pk`, `paciente_email_idx` (único, `email` admite nulos) y `paciente_nombre_idx` (`nombre` es `not null`).

```
A. select count(*) from paciente where nombre like 'M%';
B. select * from paciente where upper(email) = 'BOB@MAIL.COM';
C. select nombre from paciente order by nombre desc;
D. select * from paciente where paciente_id between 100 and 120;
E. select email from paciente where email is null;
```

4.4.4. 💰 Desglose del costo: I/O y CPU

¿Qué indica exactamente el número entre paréntesis de la columna `Cost (%CPU)`?

El costo de cada operación se compone de dos partes, ambas expresadas en la misma unidad (equivalentes a lecturas de un solo bloque):

- Costo de I/O. Proporcional al número de bloques que la operación debe leer. Las lecturas multibloque de un *full scan* se ponderan con el tiempo de una lectura multibloque.
- Costo de CPU. Proporcional a los ciclos de procesador estimados para recorrer los bloques, evaluar predicados, ordenar, agrupar, etc. El optimizador convierte esos ciclos a unidades de costo empleando las estadísticas del sistema (velocidad del procesador y tiempo de una lectura de un bloque).

[[keep: formula-03]]

$$costo = costo_{I/O} + costo_{CPU} \quad , \quad \%CPU = \frac{costo_{CPU}}{costo} * 100$$

[[/keep]]

El `%CPU` **no** es la cantidad de procesador que consume la operación, sino el porcentaje del costo total que corresponde a CPU. Un `%CPU` alto puede significar que el costo de CPU es grande o, simplemente, que el costo de I/O es pequeño. Por ello, dos planes de la misma sentencia pueden tener un costo de CPU muy parecido y diferir casi por completo en su costo de I/O: es el caso típico al comparar el acceso a una tabla contra el acceso a una estructura más pequeña que contiene los mismos datos (por ejemplo, un índice), como se ilustra en el primer ejemplo de esta sección, en el que el costo de CPU es incluso despreciable.
Como el plan redondea el `%CPU` a entero, el desglose calculado a partir de él es aproximado. Los valores exactos se obtienen de la tabla `plan_table`:

Columna de plan_table	Contenido
`cost`	Costo total de la operación (el que muestra el plan).
`io_cost`	Costo de I/O, en las mismas unidades que `cost`.
`cost - io_cost`	Costo de CPU, en las mismas unidades que `cost`.
`cpu_cost`	Ciclos de procesador estimados. Está en unidades internas y **no** es comparable directamente con `cost`.

💻 Ejemplo: desglose del costo de I/O y CPU

Retomar el conteo de registros de `paciente` de la sección *Index fast full scan*, resuelto leyendo el índice `paciente_pk`, y compararlo contra el mismo conteo forzando un *full table scan* con el hint `full`:

```
explain plan set statement_id = 'con_ffs' for
  select count(*) from paciente;

explain plan set statement_id = 'con_full' for
  select /*+ full(p) */ count(*) from paciente p;

select plan_table_output
from table(dbms_xplan.display('PLAN_TABLE', 'con_full', 'TYPICAL'));
```

Desglose del costo de ambos planes:

```
set linesize 100
col statement_id format a15
column operacion format a20
select statement_id, id, operation || ' ' || options as operacion,
  cost, io_cost, cost - io_cost as cpu_cost_aprox
from plan_table
where statement_id in ('con_ffs', 'con_full')
and cost is not null
order by statement_id, id;
```

Plan un index fast full scan:

```
PLAN_TABLE_OUTPUT
--------------------------------------------------------------------------------
Plan hash value: 2895057998
-------------------------------------------------------------------------------
| Id  | Operation             | Name        | Rows  | Cost (%CPU)| Time     |
-------------------------------------------------------------------------------
|   0 | SELECT STATEMENT      |             |     1 |    13   (0)| 00:00:01 |
|   1 |  SORT AGGREGATE       |             |     1 |            |          |
|   2 |   INDEX FAST FULL SCAN| PACIENTE_PK | 15000 |    13   (0)| 00:00:01 |
-------------------------------------------------------------------------------
9 rows selected.
```

Plan forzando un full table scan:

```
PLAN_TABLE_OUTPUT
------------------------------------------------------------------------------
Plan hash value: 66640179
-------------------------------------------------------------------------
| Id  | Operation          | Name     | Rows  | Cost (%CPU)| Time     |
-------------------------------------------------------------------------
|   0 | SELECT STATEMENT   |          |     1 |   477   (0)| 00:00:01 |
|   1 |  SORT AGGREGATE    |          |     1 |            |          |
|   2 |   TABLE ACCESS FULL| PACIENTE | 15000 |   477   (0)| 00:00:01 |
-------------------------------------------------------------------------
```

Desglose del costo de I/O y CPU:

[[keep: costos-01]]

```
STATEMENT_ID            ID OPERACION                 COST    IO_COST CPU_COST_APROX
--------------- ---------- -------------------- ---------- ---------- --------------
con_ffs                  0 SELECT STATEMENT             13         13              0
con_ffs                  2 INDEX FAST FULL SCAN         13         13              0
con_full                 0 SELECT STATEMENT            477        477              0
con_full                 2 TABLE ACCESS FULL           477        477              0
```

[[/keep]]

Plan	Operación de acceso	Costo total	%CPU	Costo de I/O	Costo de CPU
Con índice	`INDEX FAST FULL SCAN`	13	0	13	0
Con hint `full`	`TABLE ACCESS FULL`	477	0	477	0

- El costo es prácticamente todo I/O. En ambos planes `cost` coincide con `io_cost`: contar 15 000 registros requiere tan poco procesador, comparado con el costo de leer los bloques, que su costo se redondea a cero. Por eso ambos planes muestran `(0)` en la columna `%CPU`.
- La diferencia entre los planes está en el I/O. Los dos recorren los mismos 15 000 registros, pero el *index fast full scan* lee únicamente los bloques del índice de la llave primaria, mientras que el *full table scan* lee todos los bloques de la tabla hasta la HWM. Como la tabla ocupa muchos más bloques que el índice, el costo del plan con *full table scan* es aproximadamente 37 veces mayor.
- Conclusión. En sentencias como esta, que leen datos sin ordenarlos, agruparlos ni evaluar funciones costosas, el costo de CPU es despreciable y comparar planes equivale a comparar cuántos bloques lee cada uno. El costo de CPU cobra relevancia en operaciones que procesan muchos registros en memoria (ordenamientos, agregaciones, joins) o cuando el costo de I/O es muy pequeño, como ocurre cuando los datos se leen de estructuras en memoria.

💻 Ejemplo: operaciones que agregan costo de CPU

En el siguiente plan, las operaciones de agrupación y ordenamiento no leen bloques adicionales, pero sí procesan en memoria los registros que reciben:

```
explain plan set statement_id = 'cpu_1' for
  select /*+ full(p) */ nombre, count(*)
  from paciente p
  group by nombre
  order by 2 desc;

set linesize 100
col statement_id format a15
column operacion format a20
select statement_id, id, operation || ' ' || options as operacion,
  cost, io_cost, cost - io_cost as cpu_cost_aprox
from plan_table
where statement_id = 'cpu_1'
and cost is not null
order by id;
```

[[keep: costos-02]]

```
STATEMENT_ID            ID OPERACION                    COST    IO_COST CPU_COST_APROX
--------------- ---------- -------------------- ---------- ---------- --------------
cpu_1                    0 SELECT STATEMENT            480        477              3
cpu_1                    1 SORT ORDER BY               480        477              3
cpu_1                    2 HASH GROUP BY               480        477              3
cpu_1                    3 TABLE ACCESS FULL           477        477              0
```

[[/keep]]

- Recordando que el costo de cada operación es acumulado (incluye el de sus hijas), el costo de I/O se mantiene en 477 en todo el árbol: solo el *full table scan* lee bloques.
- El costo de CPU aparece a partir de la operación 2: el `hash group by` y el `sort order by` procesan los registros en memoria y elevan el costo de 477 a 480 sin realizar lecturas adicionales.
- Con tablas de algunos miles de registros, este costo sigue siendo pequeño frente al I/O. El componente de CPU se vuelve significativo con volúmenes mayores, con operaciones más intensivas (ordenamientos de millones de registros, joins grandes) o cuando el I/O es muy reducido.

📝 **Nota**
🔗 **Gancho hacia la parte 4.** En una consulta distribuida, la función de costo agrega un tercer componente, el **costo de comunicación**, que depende del volumen de datos que viaja entre sitios. Del mismo modo que aquí se separa el costo de I/O del de CPU, en la parte 4 se separa el costo de comunicación para decidir dónde conviene ejecutar cada operación.

4.5. 📚 Referencias

[[keep: bio]]

- Özsu, M. T. y Valduriez, P. *Principles of Distributed Database Systems*. Springer.
- Oracle. *Oracle Database SQL Tuning Guide*, capítulos de optimizador, planes de ejecución y métodos de acceso.
- Oracle Optimizer Blog. *How to Generate a Useful SQL Execution Plan*. https://blogs.oracle.com/optimizer/post/how-to-generate-a-useful-sql-execution-plan
- Oracle Optimizer Blog. *SQL Analysis Report in 23c Free*. https://blogs.oracle.com/optimizer/post/sql-analysis-report-in-23c-free
- Oracle Connect. *How to Read an Execution Plan*. https://blogs.oracle.com/connect/post/how-to-read-an-execution-plan
- Lewis, J. *Execution Plans, Part 3: The Rule* y *Part 8: Cost, Time, etc.* Simple Talk. https://www.red-gate.com/simple-talk/databases/oracle-databases/execution-plans-part-3-the-rule/

[[/keep]]